

Design Document: Car Dashboard

1. Project Overview

This project involves the development of a digital automotive dashboard (Automotive Cluster) designed for the BeagleBone Black hardware platform. The system displays critical vehicle information, such as speed (throttle), battery life, and system warnings. As the hardware lacks a built-in display, the dashboard operates as a "headless" system, accessible remotely via a VNC server.

2. Linux Build & Boot System

The system follows a specific sequence to power on and initialize the dashboard software. This ensures that all hardware components and background processes are fully operational before the user sees the interface.

2.1 Boot Sequence Components

The boot process moves through several stages of hardware and software initialization:

- **Initial Power-On & Early Boot ROM:** The very first step where the hardware receives power.
- **MLO & U-Boot Image:** These are the primary and secondary bootloaders that prepare the hardware to load the operating system.
- **Kernel & Device Tree:** The Linux kernel image is loaded. The "Device Tree" (merging DTB with DTBO) tells the kernel exactly what hardware parts are connected to the BeagleBone Black.
- **Initramfs:** A temporary file system used during the early stages of booting to help transition to the final system.
- **Root Filesystem (Rootfs) Mount:** The kernel connects to the main storage to find the system files and start the main control program (systemd).

2.2 Systemd Management

Once Linux is running, **systemd** (Process ID 1) manages the different services. It uses a "graphical.target" to ensure the following services start in the correct order:

- **bt-gateway.service:** Starts the communication link between the car's sensors and the computer.
- **automotive-dashboard.service:** Executes the primary graphical dashboard interface.
- **Dependencies:** Logical lines ensure that the graphical interface does not start until the network and hardware communication services are ready.

3. System Architecture

The system is divided into parts that pass information from the car's sensors to the screen.

- **Target Hardware:** BeagleBone Black (ARMv7, 1GHz Processor, 512MB RAM).
- **Operating System:** Debian Linux.
- **Data Path:**
 - a. Sensors send raw data via a serial port (`/dev/ttyS1`).
 - b. A Python script (**bt_gateway.py**) cleans this data.
 - c. The data is sent through a **UNIX domain socket**.
 - d. A C++ backend (**LocalSocketServer**) receives the data.
 - e. The **Dashboard API** updates the values.
 - f. The **QML Frontend** displays the updated values to the user.

4. Software Subsystems

4.1 Serial-to-Socket Gateway (Python)

The `bt_gateway.py` script acts as a bridge. It listens to the serial port at 9600 baud, organizes the data into commands, and sends them to the dashboard software. It will automatically try to reconnect every 2 seconds if the connection is lost.

4.2 Backend Integration (C++)

The C++ layer acts as the "brain" of the application:

- **LocalSocketServer:** Listens for incoming data on the internal socket. It cleans up the text and handles special characters.
- **Dashboard API:** Serves as the single source of truth for the vehicle state. It maintains data such as gear position, battery level, and warning status, updating the display only when values change to optimize power consumption.

4.3 Frontend Presentation (Qt/QML)

The user interface is built using Qt Quick/QML. It is designed to be responsive and easy to read.

- **Main Layout:** The dashboard is designed at 1024x600 but automatically scales to fit an 800x480 screen.
- **Gauges:** Circular dials for Throttle and Battery.
- **Car Status:** A visual map of the car showing if doors are open, if brakes are pressed, or if lights are on.
- **Gear Selector:** Shows P (Park), R (Reverse), N (Neutral), and D (Drive) with a cyan highlight for the active gear.
- **Header:** Shows the date, time, and temperature (with a snow icon if it's freezing).

5. Build and Performance Optimization

5.1 Linux Build System (CMake)

The project is compiled using **CMake**. It links the necessary Qt modules and packages all images and icons directly into the final program file so they are easy to move to the hardware.

5.2 Performance Tweaks

Since the BeagleBone Black has limited power, the following optimizations are used:

- **Draw Rate Limit:** The screen only updates 10 times per second (10 FPS). This prevents the computer from getting too hot or crashing.
- **Resolution:** Fixed at 800x480 to keep the load low.
- **Animation Control:** During the initial boot "sweep" animation, smooth transitions are temporarily disabled to ensure the gauges move accurately without lag.

6. Deployment

To get the software onto the car dashboard, an automated process is used:

1. **Stop Services:** The existing dashboard is turned off so files can be updated.
2. **Transfer Files:** The new program and scripts are sent to the board using SSH and SCP.
3. **Setup Script:** A script (`setup_board.sh`) moves files to the right folders and sets the correct permissions.
4. **Restart:** The system reloads the settings and starts the new dashboard and gateway services automatically.